



Índices jerárquicos en Pandas y Pivot

Indexación jerárquica (MultiIndex)

La **indexación jerárquica** o multinivel abre posibilidades a análisis y manipulaciones de datos sofisticadas, especialmente para trabajar con datos de dimensiones superiores. Hasta ahora, nos hemos enfocado principalmente en datos unidimensionales y bidimensionales, almacenados en objetos del tipo Serie y DataFrame de pandas, respectivamente, pero en muchas situaciones es útil ir más allá y trabajar con datos de más de dos dimensiones, es decir, **datos indexados** por más de una o dos claves.

Esto puede sonar algo confuso al inicio, por eso revisaremos algunos ejemplos para aclarar a qué nos referimos exactamente con indexación “jerárquica” y cómo podemos integrar esto con la funcionalidad de indexación básica de pandas.

Empecemos con los ejemplos.

Supongamos que queremos tener registros de los datos sobre la población de 3 estados de los Estados Unidos en dos años diferentes. Podríamos usar simplemente tuplas de Python para representar los índices y crear una serie:

```
index = [('California', 2000), ('California', 2010),  
        ('New York', 2000), ('New York', 2010),  
        ('Texas', 2000), ('Texas', 2010)]
```

```
populations = [33871648, 37253956,  
               18976457, 19378102,  
               20851820, 25145561]
```

```
pop = pd.Series(populations, index=index)
```

```
pop
```

Primero, creamos una lista de tuplas con los valores que queremos almacenar y la guardamos en la variable `index`. A continuación, creamos una lista con los valores de la población de esos estados en esos años. Finalmente, creamos la serie y la almacenamos en la variable `pop`.

Al desplegar el contenido de la variable `pop`, vemos que contiene una serie que se ve así:

```
0  
(California, 2000) 33871648  
(California, 2010) 37253956  
(New York, 2000) 18976457  
(New York, 2010) 19378102  
(Texas, 2000) 20851820  
(Texas, 2010) 25145561  
dtype: int64
```

Con esta forma de indexado podemos obtener partes de la serie usando el índice de tuplas que estamos utilizando:

```
pop[('California', 2010):('Texas', 2000)]
```

```
0  
(California, 2010) 37253956  
(New York, 2000) 18976457  
(New York, 2010) 19378102  
(Texas, 2000) 20851820  
dtype: int64
```

Pero, ¿qué pasaría si queremos consultar todos los valores de un año específico, por ejemplo, el 2010? Se puede hacer, pero no de una manera sencilla:

```
pop[[i for i in pop.index if i[1] == 2010]]
```

```
0
(California, 2010) 37253956
(New York, 2010) 19378102
(Texas, 2010) 25145561
dtype: int64
```

O de una manera más simple:

```
print(pop[("California", 2010)])
print(pop[("New York", 2010)])
print(pop[("Texas", 2010)])

37253956
19378102
25145561
```

Pero esto no es nada apropiado, sobre todo si vamos a trabajar con conjuntos de datos grandes, ya que nuestro índice contiene varios niveles (la etiqueta del estado y el valor del año).

Aquí es donde entra la ventaja de los **multiíndices**. El índice con el que estamos trabajando, representado como una tupla, en esencia es un multiíndice rudimentario, y el tipo de dato `MultiIndex` de pandas nos facilita este tipo de operaciones.

A partir de las tuplas que estamos usando como índices, podemos crear un `MultiIndex` de la siguiente manera:

```
index = pd.MultiIndex.from_tuples(index)
index
```

```
MultiIndex([('California', 2000),
            ('California', 2010),
            ('New York', 2000),
            ('New York', 2010),
            ('Texas', 2000),
            ('Texas', 2010)],
           )
```

Observa que el `MultiIndex` contiene múltiples niveles de indexación: en este caso, los nombres de los estados y los años. Si reindexamos nuestra serie con este multiíndice, observamos la representación jerárquica de los datos:

```
pop = pop.reindex(index)
pop
```

		0
California	2000	33871648
	2010	37253956
New York	2000	18976457
	2010	19378102
Texas	2000	20851820
	2010	25145561

dtype: int64

En este resultado podemos ver que ahora tenemos tres columnas: las dos primeras muestran los valores del `Multiindex`, mientras que la tercera muestra los datos. Algo interesante a tomar en cuenta es que las filas pares (la segunda, cuarta y sexta) no tienen un valor en la primera columna. Esto se debe a la representación del `Multiindex`: los valores en blanco indican que son iguales a los de la fila anterior.

Ahora sí, para ver la información cuyo segundo valor del índice es 2010, podemos hacerlo de la siguiente manera:

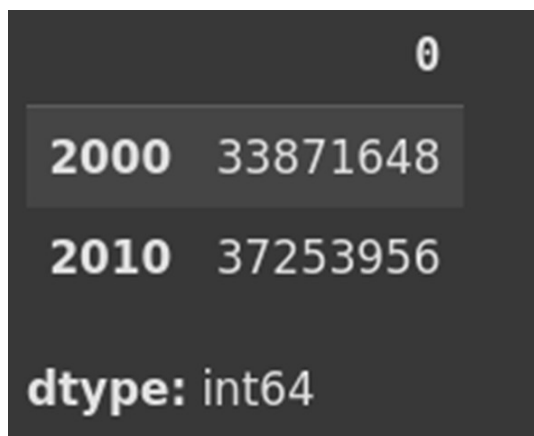
```
pop.loc[:, 2010]
```

		0
California		37253956
New York		19378102
Texas		25145561

dtype: int64

De manera similar, si queremos ver los valores para un estado específico, podemos utilizar:

```
pop.loc["California",:]
```



```
0
2000  33871648
2010  37253956
dtype: int64
```

Observa que la sintaxis es mucho más simple y la operación mucho más eficiente. El resultado es un arreglo con un sólo índice que contiene los datos que nos interesan.

Hasta este momento hemos trabajado con series, pero seguramente te estás preguntando por qué no trabajar con esta información de manera tabular, es decir, en un DataFrame, en el que podríamos tener etiquetas para los índices de las filas y de las columnas. Por ejemplo, podríamos tener como etiquetas de las filas los nombres de los estados, y como etiquetas de las columnas, los años que estamos trabajando.

Pandas tiene funcionalidad para convertir esta serie con un `MultiIndex` en un `DataFrame`. El método `unstack()` puede ayudarnos a convertir una serie con un `MultiIndex` en un `DataFrame`:

```
pop_df = pop.unstack()  
pop_df
```

	2000	2010
California	33871648	37253956
New York	18976457	19378102
Texas	20851820	25145561

Y si hay una forma de pasar de la serie con `MultiIndex` a un `DataFrame`, también tenemos una forma de pasar del `DataFrame` a una serie con `MultiIndex`:

```
pop_df.stack()
```

		0
California	2000	33871648
	2010	37253956
New York	2000	18976457
	2010	19378102
Texas	2000	20851820
	2010	25145561
		dtype: int64

Podemos ver que el uso de multiíndices nos facilita la consulta de información. Veamos otro ejemplo para resaltar aún más su importancia.

Supongamos que queremos agregar otra columna a nuestra información de la población de los estados, por ejemplo, la población menor de 18 años. Usando `MultiIndex`, podemos simplemente agregar otra columna a un `DataFrame`.

```
pop_df = pd.DataFrame({'total': pop,  
                        'under18': [9267089, 9284094,  
                                    4687374, 4318033,  
                                    5906301, 6879014] })
```

`pop_df`

		total	under18
California	2000	33871648	9267089
	2010	37253956	9284094
New York	2000	18976457	4687374
	2010	19378102	4318033
Texas	2000	20851820	5906301
	2010	25145561	6879014

Con esta instrucción asignamos los valores que teníamos previamente guardados a la columna `total` y agregamos la nueva columna `under18`. De esta forma, podríamos preguntarnos, ¿qué sentido tiene entonces trabajar con multiíndices? La respuesta es simple: así como previamente trabajábamos con información bidimensional en una serie con multiíndices, ahora podemos trabajar con información de tres o más dimensiones en una serie o en un `DataFrame`. Cada nivel adicional en un multiíndice representa una dimensión más de información.

Además, podemos utilizar toda la funcionalidad de pandas. Por ejemplo, calculemos el porcentaje de la población menor de 18 para cada estado por año:

```
(pop_df["under18"] / pop_df['total'])
```

0		
California	2000	0.273594
	2010	0.249211
New York	2000	0.247010
	2010	0.222831
Texas	2000	0.283251
	2010	0.273568
dtype: float64		

Incluso, esta información la podemos convertir en un DataFrame, como lo hicimos previamente con el método `unstack()`, para manipular de manera más fácil y rápida la información multidimensional con la que estemos trabajando.

```
(pop_df["under18"] / pop_df['total']).unstack()
```

	2000	2010
California	0.273594	0.249211
New York	0.247010	0.222831
Texas	0.283251	0.273568

La manera más fácil de construir una serie o un DataFrame con un `MultiIndex` es simplemente pasar una lista de dos o más arreglos de índices al constructor del DataFrame, por ejemplo:

```
df = pd.DataFrame(np.random.rand(4, 2),  
                  index=[['a', 'a', 'b', 'b'], [1, 2, 1, 2]],  
                  columns=['data1', 'data2'])
```

df

		data1	data2
a	1	0.573826	0.266158
	2	0.440199	0.622290
b	1	0.503216	0.563156
	2	0.998552	0.806069

Veamos esto por pasos. Si sólo creamos el DataFrame con la instrucción:

```
df = pd.DataFrame(np.random.rand(4, 2))
```

	0	1
0	0.562203	0.552725
1	0.490092	0.398520
2	0.219268	0.668146
3	0.711014	0.630616

Nos genera un DataFrame de dos dimensiones, de 4 renglones y dos columnas con valores aleatorios.

Al agregar los índices:

```
df = pd.DataFrame(np.random.rand(4, 2),  
                  index=[['a', 'a', 'b', 'b'], [1, 2, 1, 2]])
```

Nos asigna un `MultiIndex` al DataFrame:

		0	1
a	1	0.138919	0.261356
	2	0.234496	0.728944
b	1	0.058519	0.135479
	2	0.520159	0.050375

Al ejecutar el código completo, reemplaza las etiquetas de columnas de 0, 1 a data1, data2

```
df = pd.DataFrame(np.random.rand(4, 2),  
                  index=[['a', 'a', 'b', 'b'], [1, 2, 1, 2]],  
                  columns=['data1', 'data2'])
```

df

		data1	data2
a	1	0.149500	0.474886
	2	0.635917	0.267069
b	1	0.684142	0.014385
	2	0.028172	0.079831

Obviamente, los valores cambian porque, cada vez que ejecutamos el código, se generan nuevos valores aleatorios.

De manera similar, si pasamos como argumento al constructor de la serie un diccionario donde las llaves son tuplas, pandas reconocerá esto automáticamente y usará un `MultiIndex` para crear la serie.

```
data = {('California', 2000): 33871648,  
        ('California', 2010): 37253956,  
        ('Texas', 2000): 20851820,  
        ('Texas', 2010): 25145561,  
        ('New York', 2000): 18976457,  
        ('New York', 2010): 19378102}
```

```
df1 = pd.Series(data)  
df1
```

0		
California	2000	33871648
	2010	37253956
Texas	2000	20851820
	2010	25145561
New York	2000	18976457
	2010	19378102
dtype: int64		

Y como ya vimos, podemos aplicar `unstack()` para convertirlo en DataFrame:

```
df2 = df1.unstack()  
df2
```

	2000	2010
California	33871648	37253956
New York	18976457	19378102
Texas	20851820	25145561

Para mayor flexibilidad, podemos utilizar dos constructores explícitos diferentes de `MultiIndex`:

`pd.MultiIndex.from_arrays` y `pd.MultiIndex.from_tuples`.
Veamos un par de ejemplos.

Podemos construir el `MultiIndex` a partir de una lista simple de arreglos, dando los valores de los índices dentro de cada nivel:

```
pd.MultiIndex.from_arrays(['a', 'a', 'b', 'b'], [1, 2, 1, 2])
```

```
MultiIndex([('a', 1),  
            ('a', 2),  
            ('b', 1),  
            ('b', 2)],  
           )
```

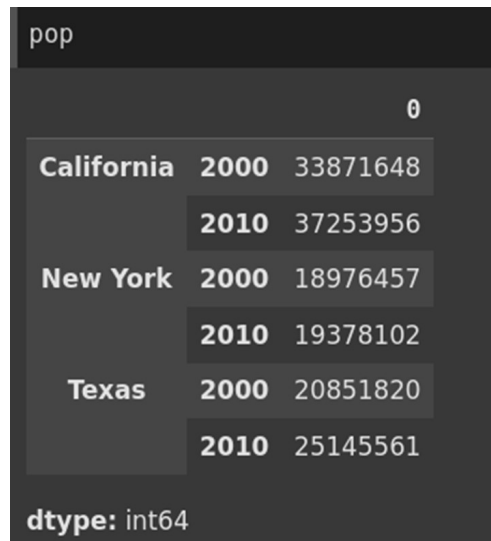
También podemos construir el `MultiIndex` a partir de una lista de tuplas, dando los valores de los múltiples índices:

```
pd.MultiIndex.from_tuples([('a', 1), ('a', 2), ('b', 1),  
                           ('b', 2)])
```

```
MultiIndex([('a', 1),  
            ('a', 2),  
            ('b', 1),  
            ('b', 2)],  
           )
```

En ocasiones, puede ser útil etiquetar o poner nombres a los niveles del `MultiIndex`. Esto podemos hacerlo pasando el argumento `names` a cualquiera de los constructores del `MultiIndex`, o asignándole nombres después de crearlo.

Por ejemplo, si nuestra serie con `MultiIndex` de los estados, que llamamos `pop`, se encuentra de la siguiente manera:



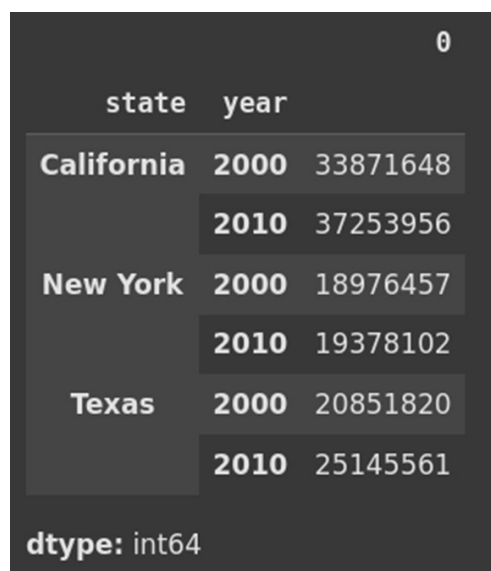
```
pop
```

0		
California	2000	33871648
	2010	37253956
New York	2000	18976457
	2010	19378102
Texas	2000	20851820
	2010	25145561

dtype: int64

Podemos asignarle los nombres de `state` y `year` al `MultiIndex` de la siguiente manera:

```
pop.index.names = ['state', 'year']  
pop
```



```
pop
```

state	year	0
California	2000	33871648
	2010	37253956
New York	2000	18976457
	2010	19378102
Texas	2000	20851820
	2010	25145561

dtype: int64

Todo esto que hemos hecho con los índices nos da nuevas posibilidades para trabajar con el análisis de datos, pero aún nos falta trabajar con las columnas.

Así como las filas de un DataFrame pueden tener varios niveles de índices, las columnas también. Veamos un ejemplo para ilustrar esto.

Supongamos que tenemos un conjunto de datos que representa alguna especie de registros médicos.

```
# índices y columnas jerárquicos
index = pd.MultiIndex.from_product([[2013, 2014], [1, 2]],
                                   names=['year', 'visit'])
columns = pd.MultiIndex.from_product([
    ['Ana', 'Luis', 'Pedro'],
    ['HR', 'Temp']],
                                   names=['subject', 'type'])
# insertemos datos ficticios
data = np.round(np.random.randn(4, 6), 1)
data[:, ::2] *= 10
data += 37
# create the DataFrame
health_data = pd.DataFrame(data, index=index,
                           columns=columns)
health_data
```

		subject Ana		Luis		Pedro	
		type HR	Temp	HR	Temp	HR	Temp
year	visit						
2013	1	59.0	37.1	47.0	37.9	30.0	37.0
	2	45.0	38.8	35.0	37.7	42.0	36.5
2014	1	37.0	37.2	35.0	36.0	35.0	38.8
	2	39.0	39.3	44.0	38.2	37.0	36.5

Vamos por partes. Si no lograste comprender por completo qué fue lo que pasó, te lo explicamos:

Primero, creamos un `MultiIndex` para las filas. Usamos el producto de dos arreglos: `[2013, 2014]` y `[1, 2]`, lo que nos generará las tuplas `(2013, 1)`, `(2013, 2)`, `(2014, 1)` y `(2014, 2)`. A estas tuplas, que usaremos como índices, les pondremos los nombres `year` y `visit`, para indicar que fue la visita 1 del 2013, la visita 2 del 2013, la visita 1 del 2014 y la visita 2 del 2014.

```
index = pd.MultiIndex.from_product(  
    [[2013, 2014], [1, 2]],  
    names=['year', 'visit'])  
index
```

```
MultiIndex([(2013, 1),  
            (2013, 2),  
            (2014, 1),  
            (2014, 2)],  
           names=['year', 'visit'])
```

Luego, creamos otro `MultiIndex` para las columnas, de manera similar, usando `pd.MultiIndex.from_product()`, pasándole los arreglos `['Ana', 'Luis', 'Pedro']` y `['HR', 'Temp']`, lo que nos generará las tuplas:

```
('Ana', 'HR'), ('Ana', 'Temp'), ('Luis', 'HR'), ('Luis',  
'Temp'), ('Pedro', 'HR') y ('Pedro', 'Temp').
```

Esto significa que tenemos a tres personas y dos datos por persona: la frecuencia cardíaca (`HR` – *heart rate*) y la temperatura (`Temp`).

También asignamos los nombres `subject` y `type` para indicar de quién es el dato y el tipo de dato.

```
columns = pd.MultiIndex.from_product([
    ['Ana', 'Luis', 'Pedro'],
    ['HR', 'Temp']],
    names=['subject', 'type'])
columns
```

```
MultiIndex([( 'Ana',  'HR'),
            ( 'Ana',  'Temp'),
            ( 'Luis',  'HR'),
            ( 'Luis',  'Temp'),
            ( 'Pedro', 'HR'),
            ( 'Pedro', 'Temp')],
            names=['subject', 'type'])
```

En las siguientes líneas es donde se generan los valores aleatorios de 4 renglones y 6 columnas con un punto decimal. Luego, se ajustan para que parezcan valores válidos de frecuencia cardíaca y temperatura.

```
data = np.round(np.random.randn(4, 6), 1)
data[:, :2] *= 10
data += 37
data
```

Finalmente, construimos el `DataFrame` con los datos generados. Como índices, asignamos el `MultiIndex` llamado `index`, y como nombres de las columnas, asignamos el `MultiIndex` `columns`.

```
health_data = pd.DataFrame(data, index=index, columns=columns)
```

Bien, ya con nuestro DataFrame con `MultiIndex` tanto en filas como en columnas, podemos hacer consultas específicas de una manera muy simple y rápida.

		subject		Ana		Luis		Pedro	
		type		HR	Temp	HR	Temp	HR	Temp
year	visit								
2013	1			59.0	37.1	47.0	37.9	30.0	37.0
	2			45.0	38.8	35.0	37.7	42.0	36.5
2014	1			37.0	37.2	35.0	36.0	35.0	38.8
	2			39.0	39.3	44.0	38.2	37.0	36.5

Por ejemplo, si queremos ver los registros de Luis, podemos hacerlo tan simple como:

```
health_data['Luis']
```

		type	HR	Temp
year	visit			
2013	1		47.0	37.9
	2		35.0	37.7
2014	1		35.0	36.0
	2		44.0	38.2

Podemos consultar información usando más de un valor para los `MultiIndex`. Por ejemplo, para consultar la frecuencia cardíaca de Luis en todas sus visitas, podemos hacerlo así:

```
health_data[('Luis', 'HR')]
```

		Luis
		HR
year	visit	
2013	1	47.0
	2	35.0
2014	1	35.0
	2	44.0

dtype: float64

El uso de **filas y columnas jerárquicas** puede ser muy conveniente en escenarios donde se tengan **registros complicados**, por ejemplo, aquellos que contengan múltiples mediciones etiquetadas en múltiples momentos para muchos sujetos (personas, países, ciudades, entre otros).

Antes de continuar...

Como pudiste ver, el manejo de datos jerárquicos en pandas, mediante estructuras como `MultiIndex`, abre un abanico de posibilidades para organizar, consultar y analizar información compleja de forma eficiente y estructurada. Ya sea que trabajemos con series o DataFrames, contar con varios niveles de índices nos permite representar datos de múltiples dimensiones sin **perder claridad ni funcionalidad**. Además, herramientas como `unstack()` o el uso de índices jerárquicos en columnas nos facilitan transformar y visualizar los datos según nuestras necesidades. Dominar estas técnicas es clave para enfrentar con éxito escenarios reales en análisis de datos, donde la información rara vez es plana o simple.

Haz la siguiente reflexión:

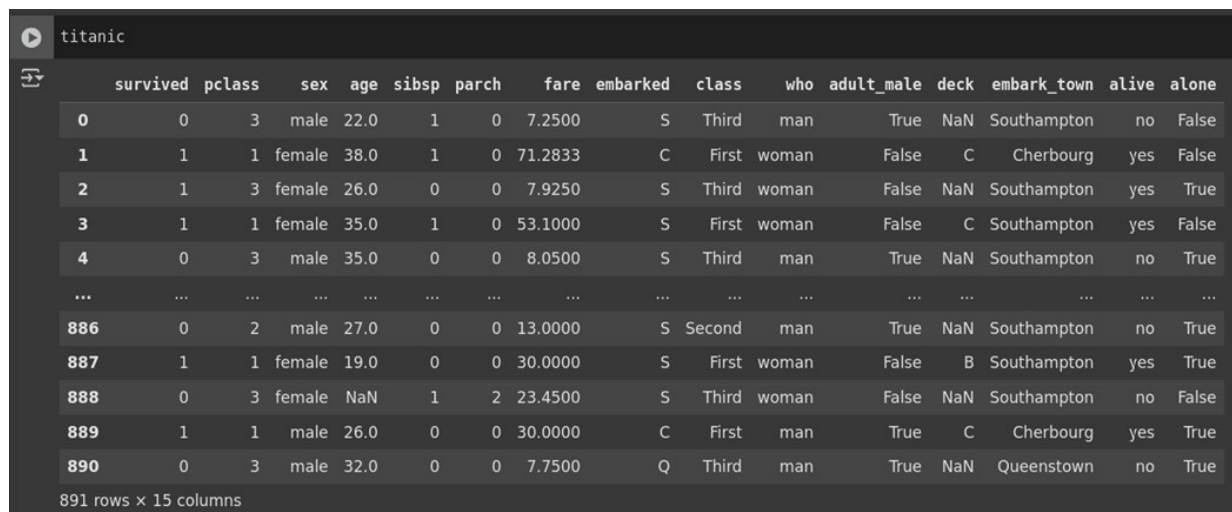
- ¿Cómo podría el uso de `MultiIndex` mejorar la organización de los datos en tus proyectos actuales?
- ¿Qué ventajas encuentras al tener varios niveles de índices frente a un enfoque más tradicional de índices planos?
- ¿En qué situaciones específicas crees que un `MultiIndex` podría resultar innecesario o contraproducente?

Tablas dinámicas (*pivot tables*)

El concepto de **tablas dinámicas** o ***pivot tables*** es muy usado en el contexto del análisis de datos; son una forma muy práctica de **mostrar resúmenes de la información**. Para ver cómo pandas trabaja con las tablas dinámicas, veamos un ejemplo utilizando un *dataset* que contiene información sobre los pasajeros a bordo del Titanic, incluyendo detalles demográficos, de viaje y su estado de supervivencia. Esta información está disponible para su uso en la biblioteca seaborn de Python.

Primero, importamos las bibliotecas y damos un vistazo a la estructura de este *dataset*:

```
import numpy as np
import pandas as pd
import seaborn as sns
titanic = sns.load_dataset('titanic')
titanic
```



	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	13.0000	S	Second	man	True	NaN	Southampton	no	True
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	B	Southampton	yes	True
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	False	NaN	Southampton	no	False
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	C	Cherbourg	yes	True
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	True	NaN	Queenstown	no	True

891 rows x 15 columns

En pandas, podemos agrupar datos con la función `groupby`. Esta función se puede utilizar, por ejemplo, para ver el promedio de sobrevivientes tanto de mujeres como de hombres utilizando la instrucción:

```
titanic.groupby('sex')[['survived']].mean()
```

Esta instrucción lo que hace es mostrar un resumen de nuestro DataFrame, agrupando los distintos valores que existan en la columna `'sex'`, que en este caso son `'female'` y `'male'`, y nos está mostrando los resultados de aplicar una operación a todos los datos que tengan ese valor específico. En este caso, se está utilizando la función `mean()`, es decir, un promedio.

survived	
sex	
female	0.742038
male	0.188908

En este caso, lo que nos está mostrando es que, de todos los registros que tienen el valor `'female'` en la columna `sex`, el 74.2038 tienen el valor 1 en la columna `'survived'`. De manera similar, el 18.89 de los registros que tienen el valor `'male'` en la columna `sex` tienen el valor 1 en la columna `'survived'`.

Esto nos da una buena idea de lo que pasó, pero nuestro *dataset* tiene más información, porque también nos dice en qué clase viajaba cada pasajero. Entonces, podríamos obtener el porcentaje de mujeres de primera clase que sobrevivieron, así como el porcentaje de mujeres de segunda y tercera clase que también lo hicieron, y lo mismo para los hombres.

Esto no es una tarea sencilla; sin embargo, puede hacerse con `groupby` y funciones. La instrucción quedaría así:

```
titanic.groupby(['sex', 'class'])['survived'].aggregate('mean').unstack()
```

	class	First	Second	Third
sex				
female	0.968085	0.921053	0.500000	
male	0.368852	0.157407	0.135447	

No es tan simple, ¿cierto? Recuerda que, en la sección de Material adicional, te compartimos un enlace con información de cómo trabajan las funciones de agregación en pandas y `groupby`.

Pero no es necesario complicarse. Para hacer este tipo de operaciones, tenemos las tablas dinámicas para ayudarnos.

En pandas, podemos hacer la operación anterior con una sintaxis un poco más entendible usando la instrucción `pivot_table`:

```
titanic.pivot_table('survived', index='sex', columns='class', aggfunc='mean')
```

En esta instrucción estamos pidiendo a pandas que haga una tabla dinámica (`pivot_table`), donde los valores que nos van a interesar son los valores de `'survived'`. Como último parámetro, `aggfunc='mean'` indica que utilizamos la función de agregación `mean`, es decir, queremos que nos muestre el promedio de los valores de `survived`. Si no especificamos este parámetro (`aggfunc`), por *default* utiliza la función `mean`. También le estamos indicando a la instrucción que queremos que tome como filas los valores de `'sex'` (`index = 'sex'`) y que como columnas nos muestre los posibles valores de `'class'`.

El resultado es:

class	First	Second	Third
sex			
female	0.968085	0.921053	0.500000
male	0.368852	0.157407	0.135447

En la sección de Material adicional, te compartimos la documentación oficial de pandas para que puedas consultar a detalle el funcionamiento de `pivot_table`. Mientras tanto, seguiremos revisando otros ejemplos.

Queremos ver los porcentajes de personas sobrevivientes por puerto de embarque (`embark_town`). Suena como algo que podemos hacer con relativa facilidad. Creamos la `pivot_table`, le decimos que queremos que construya la tabla de acuerdo al valor de `survived`, que como filas use el puerto de embarque (`embark_town`), y que utilice la función promedio (`mean`) de los valores para `survived`.

```
titanic.pivot_table('survived', index='embark_town', aggfunc='mean')
```

	survived
embark_town	
Cherbourg	0.553571
Queenstown	0.389610
Southampton	0.336957

La información puede sernos útil, pero podemos hacer la consulta más interesante. En el *dataset* también tenemos información de quienes iban solos (columna 'alone'), así que podemos agregar `columns=['alone']`, y ver el resultado:

```
titanic.pivot_table('survived', index='embark_town',  
                    columns=['alone'], aggfunc='mean')
```

	alone	False	True
embark_town			
Cherbourg		0.674699	0.435294
Queenstown		0.350000	0.403509
Southampton		0.462151	0.256997

De manera adicional, podemos agregar el parámetro `margins=True` para que pandas agregue una fila y una columna con los totales generales, y así obtener un resumen global automáticamente.

```
titanic.pivot_table('survived', index='embark_town',  
                    columns=['alone'], aggfunc='mean', margins=True)
```

	alone	False	True	All
embark_town				
Cherbourg		0.674699	0.435294	0.553571
Queenstown		0.350000	0.403509	0.389610
Southampton		0.462151	0.256997	0.336957
All		0.505650	0.300935	0.382452

Ahora, si queremos hacerlo aún más interesante, podemos pedir que también en las columnas se incluya la clase en la que viajaban los pasajeros.

```
titanic.pivot_table('survived', index='embark_town',
    columns=['alone', 'class'], aggfunc='mean')
```

alone	False			True		
	First	Second	Third	First	Second	Third
embark_town						
Cherbourg	0.772727	0.545455	0.571429	0.609756	0.500000	0.236842
Queenstown	0.500000	NaN	0.333333	NaN	0.666667	0.388889
Southampton	0.704918	0.652174	0.231405	0.469697	0.326316	0.168103

Podemos observar que nos está mostrando un DataFrame con columnas con jerarquías, nos está dando la información de si viajaban solos o no, y además, para cada valor, nos está mostrando la información de quienes iban en primera, segunda y tercera clase.

¿Queremos hacerlo aún más interesante? Agreguemos la columna 'sex' para que también nos separe de acuerdo al sexo:

```
titanic.pivot_table('survived', index='embark_town',
    columns=['alone', 'class', 'sex'], aggfunc='mean')
```

alone	False								True			
	First	Second	Third		First		Second	Third	First	Second	Third	
sex	female	male	female	male	female	male	female	male	female	male	female	male
embark_town												
Cherbourg	1.000000	0.473684	1.000000	0.166667	0.611111	0.500000	0.944444	0.347826	1.000000	0.250000	0.800000	0.151515
Queenstown	1.000000	0.000000	NaN	NaN	0.625000	0.100000	NaN	NaN	1.000000	0.000000	0.760000	0.068966
Southampton	0.941176	0.407407	0.923077	0.300000	0.327586	0.142857	1.000000	0.326923	0.892857	0.089552	0.466667	0.123762

Esta última instrucción nos está mostrando la información donde las columnas, como primer nivel, indican si el pasajero venía solo o no. Para cada valor, se muestra la clase en la que viajaban y, dentro de cada clase, si era mujer u hombre.

Para concluir con este tema, mencionemos que en algunas situaciones puede ser interesante dividir un conjunto de valores numéricos en intervalos. Por ejemplo, podríamos estar interesados en rangos de edad: personas menores de 12 años, adolescentes de 12 a 17 años, adultos de 18 a 60 años y personas mayores de 61 años en adelante.

Esto podemos hacerlo usando la función `cut` de pandas para dividir los valores de la edad, y así ver los porcentajes de sobrevivientes por cada uno de estos rangos de edad y en cada una de las clases. Primero, podemos consultar cuál es la persona de mayor edad en nuestro DataFrame, para decidir el valor del último intervalo:

```
max(titanic['age'])
```

80.0, y de acuerdo con eso, podemos hacer nuestra partición, y posteriormente, nuestra tabla dinámica:

```
age = pd.cut(titanic['age'], [0, 11, 17, 60, 80])
titanic.pivot_table('survived', ['sex', age], 'class')
```

		class	First	Second	Third
sex	age				
female	(0, 11]	0.000000	1.000000	0.478261	
	(11, 17]	1.000000	1.000000	0.666667	
	(17, 60]	0.973333	0.903226	0.409091	
	(60, 80]	1.000000	NaN	1.000000	
male	(0, 11]	1.000000	1.000000	0.333333	
	(11, 17]	1.000000	0.000000	0.105263	
	(17, 60]	0.411765	0.058824	0.135922	
	(60, 80]	0.083333	0.333333	0.000000	

Para terminar, veamos un ejemplo en donde se utilizan y combinan más funciones para las columnas. Por ejemplo, para cada sexo, queremos ver la

suma de sobrevivientes por clase y el promedio de la tarifa pagada (*fare*).

```
titanic.pivot_table(index='sex', columns='class',  
                    aggfunc={'survived':sum, 'fare':'mean'})
```

	fare			survived		
class	First	Second	Third	First	Second	Third
sex						
female	106.125798	21.970121	16.118810	91	70	72
male	67.226127	19.741782	12.661633	45	17	47

Antes de terminar...

Al concluir este tema, ya cuentas con herramientas poderosas para analizar y transformar datos de manera más estructurada y eficiente. Las tablas dinámicas (*pivot tables*) te dan la capacidad de resumir grandes volúmenes de datos con un par de líneas de código, generando reportes claros y útiles para la toma de decisiones.

Reflexiona a continuación:

- En un entorno profesional con datos (*marketing*, finanzas, educación, salud, etc.), ¿cómo podrían ayudarte las tablas dinámicas a analizar grandes volúmenes de información de manera rápida?
- ¿Te resultó más fácil interpretar los datos usando tablas dinámicas en lugar de una tabla plana?, ¿por qué?
- ¿Cómo cambiaría la utilidad de una tabla dinámica si los datos estuvieran mal estructurados o desorganizados?

I Material adicional

Pandas. (s.f.) *pandas.pivot_table*. Pandas. Recuperado el 30 de abril de 2025 de: https://pandas.pydata.org/docs/reference/api/pandas.pivot_table.html

VanderPlas, J. (s.f.) *Hierarchical Indexing*. [Índices Jerárquicos]. Python Data Science Handbook. Recuperado el 30 de abril de 2025 de: <https://jakevdp.github.io/PythonDataScienceHandbook/03.05-hierarchical-indexing.html>

Nota: Los enlaces a las páginas web incluidos en este Material adicional fueron consultados y verificados por última vez el 30 de abril de 2025 y su disponibilidad depende del autor.

| Glosario

MultiIndex: Estructura de pandas que permite múltiples niveles de índice en un *DataFrame* o serie, útil para organizar y acceder a datos complejos.

Indexing jerárquico: Forma avanzada de indexación que permite seleccionar datos en estructuras con múltiples niveles de índices.

Pivot Table (Tabla dinámica): Herramienta para resumir y reorganizar datos cruzando dimensiones (columnas y filas) y aplicando funciones de agregación.

aggfunc: Parámetro de `pivot_table` que define la función de agregación (como `mean`, `sum`, etc.) a aplicar a los datos resumidos.

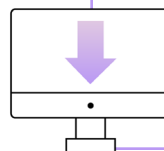
| Bibliografía

Pandas. (s.f.) *Reshaping and pivot tables*. [Reestructuración y tablas dinámicas.] Pandas. Recuperado el 30 de abril de 2025 de: https://pandas.pydata.org/docs/user_guide/reshaping.html

VanderPlas, J. (s.f.) *Pivot tables*. [Tablas dinámicas]. Python Data Science Handbook. Recuperado el 30 de abril de 2025 de: <https://jakevdp.github.io/Python-DataScienceHandbook/03.09-pivot-tables.html>

VanderPlas, J. (2016) *Python Data Science Handbook*. O'Reilly Media.

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 30 de abril de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.